Multi-Region Disaster Recovery Architecture using AWS CloudFormation and Route 53

1. Services

Lambda, API Gateway, S3, DynamoDB, Cognito

2. Definition

This project demonstrates how to design and implement a **Disaster Recovery (DR) solution** on AWS.
The application is deployed in **two different AWS regions** so that if the **primary region fails**, traffic is **automatically redirected** to the **secondary region** using Route 53 failover routing.

---

3. Scope of the Project

- Deploy identical EC2 web servers in **two AWS regions**
- Use **Infrastructure as Code (CloudFormation)** to automate resource creation
- Implement **Route 53 health checks** for monitoring
- Enable **automatic failover** during primary region failure
- Ensure **high availability and business continuity**

This project focuses on **infrastructure-level disaster recovery**, not application development.

---

4. Explanation of AWS Services (Purpose)

Amazon EC2

Used to host a simple web application (Apache web server).

AWS CloudFormation

Used to automate the creation of EC2 instances and security groups using YAML templates.

Amazon Route 53

Used for DNS management and **failover routing** between primary and secondary regions.

Route 53 Health Checks

Used to continuously monitor the health of the primary EC2 instance.

Amazon VPC (Default)

Provides networking for EC2 instances.

5. Methodology (Why I Selected This Project)

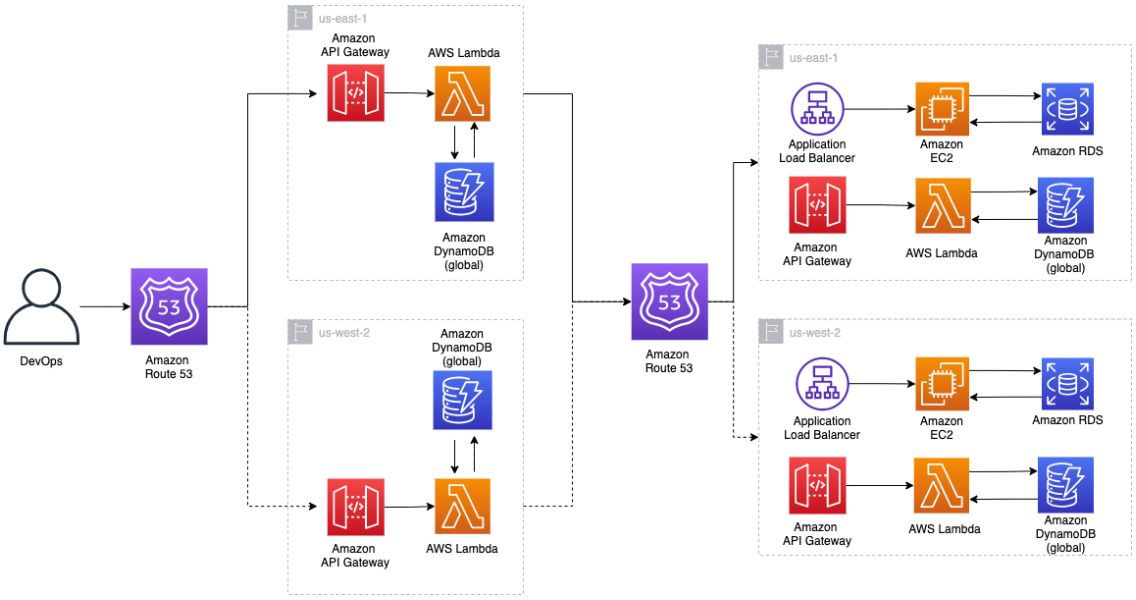
I selected this project because:

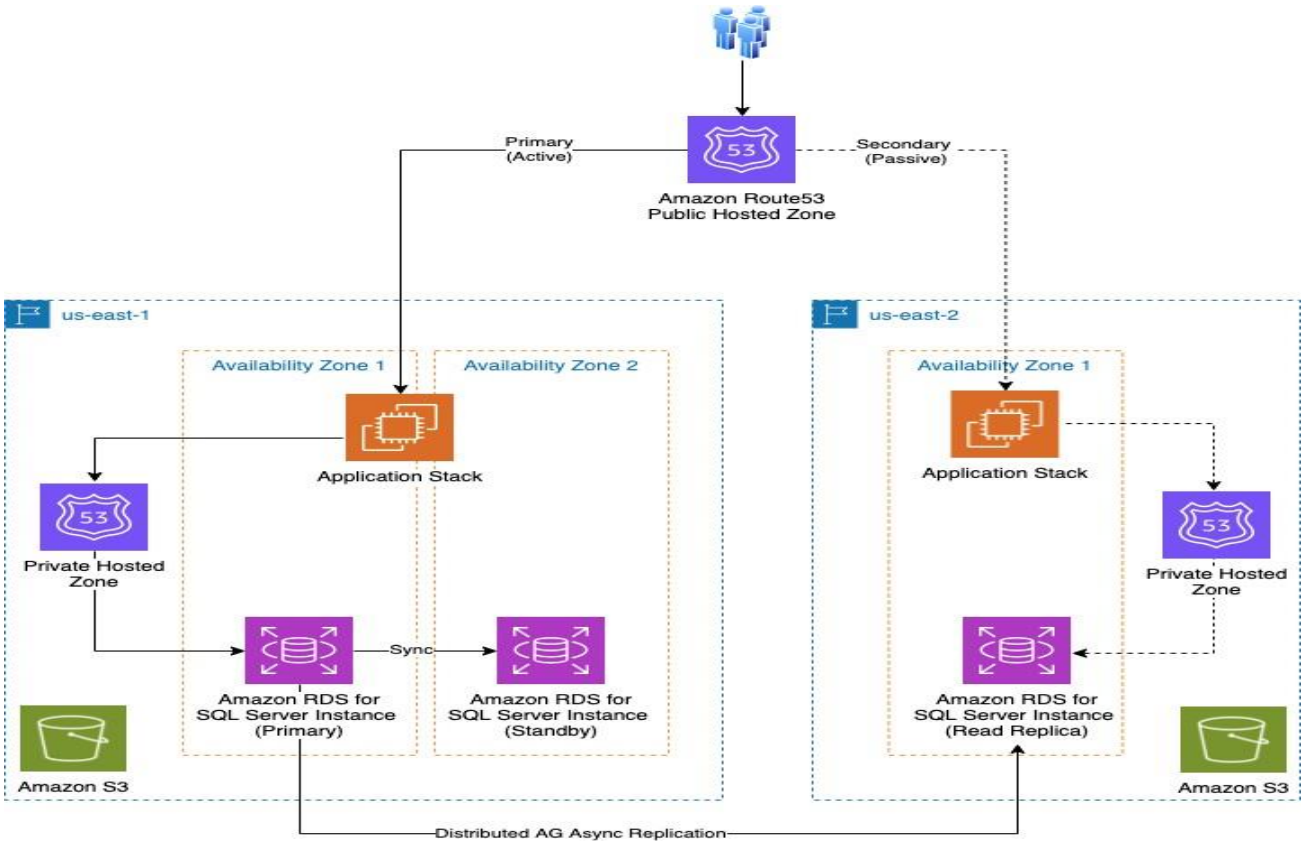
- Disaster Recovery is a **real-world production requirement**
- It demonstrates **DevOps skills**, not just theory
- It involves **multi-region architecture**
- It uses **automation (CloudFormation)** instead of manual setup
- It helps understand **DNS, health checks, and failover concepts**

This project helped me gain hands-on experience in **AWS infrastructure design and troubleshooting**.

---

6. Architecture Diagram





(Text Explanation)

Primary Region (ap-south-1 – Mumbai)

- EC2 Web Server
- Route 53 Health Check

Secondary Region (ap-southeast-1 – Singapore)

- EC2 Web Server (Standby)

Route 53

- Failover A records
- Routes traffic to Primary EC2
- Automatically switches to Secondary EC2 if Primary fails

---

7. Step-by-Step Procedure

Step 1: Select Regions

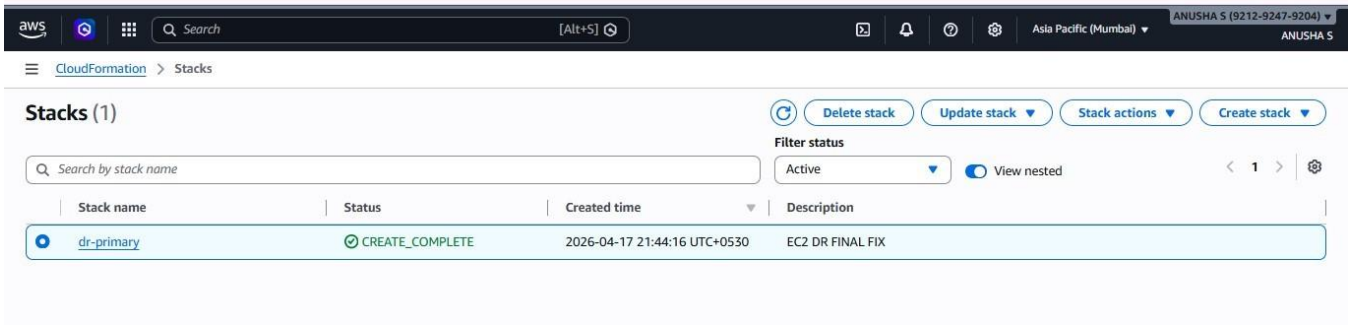
- Primary Region: Mumbai (ap-south-1)
- Secondary Region: Singapore (ap-southeast-1)

Step 2: Create CloudFormation Template

- Define EC2 instance
- Define Security Group (allow SSH & HTTP)
- Add UserData to install Apache automatically

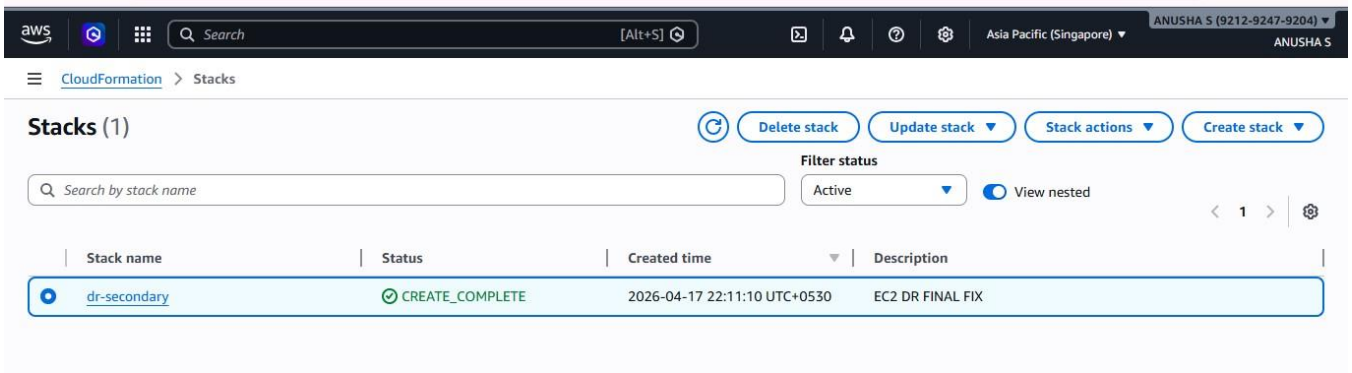
Step 3: Deploy Primary Stack

- Upload template in Mumbai region
- Verify EC2 creation
- Confirm Apache web page loads using public IP



Step 4: Deploy Secondary Stack

- Upload same template in Singapore region
- Use region-specific AMI
- Verify secondary EC2 is running and accessible

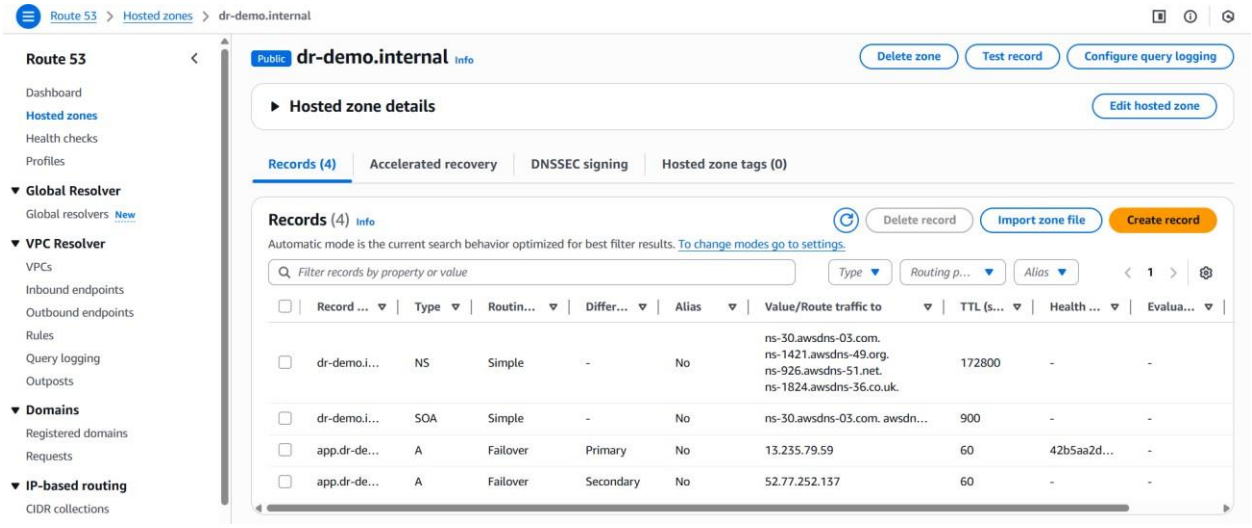


Step 5: Create Route 53 Health Check

- Health check type: HTTP
- Endpoint: Primary EC2 public IP
- Path: /
- Port: 80

Step 6: Create Route 53 Failover Records

- A record (Primary) → Primary EC2 IP + health check
- A record (Secondary) → Secondary EC2 IP
- Same record name used for both



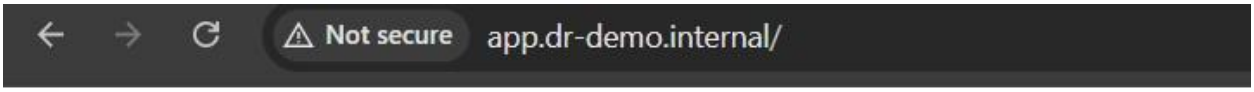
Step 7: Test Disaster Recovery

- Stop Primary EC2

- Health check turns unhealthy
- Route 53 redirects traffic to Secondary EC2
- Application remains accessible

Primary Region App Running

Disaster Recovery Web Server - Secondary



Primary Region App Running

Disaster Recovery Web Server - Secondary

8. Troubleshooting

Issue 1: CloudFormation Rollback

- **Method:** Checked Events tab
- **Solution:** Fixed YAML indentation and duplicate keys

Issue 2: EC2 Not Launching

- **Method:** Verified AMI and UserData
- **Solution:** Used correct Amazon Linux version and package manager (dnf)

Issue 3: Secondary EC2 Page Not Loading

- **Method:** SSH into instance and checked Apache status
- **Solution:** Installed and started httpd manually

Issue 4: Route 53 Domain Not Resolving

- **Method:** Tested DNS name in browser
- **Solution:** Understood .internal domains don’t resolve publicly and used hosts file for local testing

---

9. Conclusion

This project successfully demonstrates a **Disaster Recovery solution on AWS** using automation and DNS-based failover.
It ensures **high availability**, minimizes downtime, and improves system reliability.

Through this project, I gained practical experience in:

- Infrastructure as Code
- Multi-region deployment
- Route 53 failover configuration
- Troubleshooting real AWS issues

This project reflects **real-world DevOps and Cloud engineering practices**.